

面试手册

C++

把基础，读成底气。

1 个科目 / 20 道问题

回答要点 • 常见追问 • 易错点 • 参考来源

整理日期：2026-10-09

本手册总结经典面试知识点，不是全网公司面经的穷举，也不代表真实招聘频次统计。题解为独立中文整理，具体版本与实现请参考题目中的原始资料。

C++ 以 C++17/20 为主；MySQL 以 8.4 LTS InnoDB 为主；Redis 以开源版常用功能为主。目录、PDF 书签及参考链接均可点击。

阅读目录

阅读目录	2
C++	3
01. 指针、引用与对象生命周期有什么区别？	3
02. const、constexpr 和 static 分别解决什么问题？	3
03. new/delete 与 malloc/free 有什么区别？	4
04. 虚函数如何实现多态？基类析构为什么常设为虚函数？	5
05. 构造和析构顺序是什么？构造函数能调用虚函数吗？	6
06. 浅拷贝、深拷贝与 Rule of Zero/Five 怎么理解？	6
07. RAII 为什么能改善异常安全？	7
08. unique_ptr 的所有权怎样转移？release 与 reset 有何不同？	8
09. shared_ptr 的控制块与线程安全边界是什么？	8
10. weak_ptr 怎样打破循环引用？为什么使用 lock？	9
11. std::move、右值引用和完美转发有什么联系？	10
12. 模板、重载、特化与 C++20 concepts 怎么区分？	11
13. lambda 的值捕获、引用捕获与生命周期怎样判断？	11
14. vector 为什么适合默认使用？扩容与失效规则是什么？	12
15. list、deque 与 vector 如何选？	13
16. map 和 unordered_map 的性能与键规则有什么差别？	13
17. 迭代器类别、算法要求与删除遍历怎样关联？	14
18. 异常安全的基本、强与不抛保证怎么区分？	15
19. 未定义行为、越界、对齐与类型别名怎样理解？	15
20. 数据竞争、atomic、mutex 与内存序是什么关系？	16

C++

对象、所有权与现代语言机制

围绕 C++17/20 常见面试主题组织 20 题，解释语言规则、工程取舍与典型误区。草案链接用于延伸阅读，较新标准新增条款不属于本章默认口径。

01. 指针、引用与对象生命周期有什么区别？

语言基础 / 基础

指针是可以保存地址的对象，引用通常表达已绑定对象的别名。二者都不自动延长任意对象的生命，判断能否访问时要同时检查地址、对象类型和生命周期。

回答要点

1. 指针可以为空并且可以重新指向其他对象；引用初始化后不能改绑。给引用赋值是在修改它所指对象，而不是切换目标。接口中可选参数通常用指针，必需的借用参数可用引用。
2. 对象的存储存在不代表对象已经构造，也不代表它仍然存活。局部对象离开作用域、容器发生重分配或所有者销毁后，原先保存的地址都可能失效，必须从拥有者追踪有效期。
3. 语言没有规定引用必须无额外空间；编译器可以按调用约定实现引用。不能返回局部变量的指针或引用，也不能用空指针人为构造有效引用。临时量生命期延长还有特定规则。

常见追问

追问：返回 `const` 引用能避免拷贝吗？答：可以借用仍然存活的对象，但必须保证调用方使用期间拥有者存在。返回局部变量的 `const` 引用仍然悬空；仅仅加上 `const` 不能修复生命周期。小对象直接按值返回往往更清晰。

易错点

把引用描述成“永远安全的指针”会漏掉悬空引用。检查异常路径、异步回调和容器修改，因为它们都可能在使用前让被借用对象失效。

延伸阅读 / 原始参考资料

ISO C++ 工作草案 • 引用

ISO C++ 工作草案 • 生命周期

02. `const`、`constexpr` 和 `static` 分别解决什么问题？

语言基础 / 基础

`const` 约束通过某种访问路径进行修改；`constexpr` 为编译期求值提供条件；`static` 的含义依声明位置而变，可能涉及存储期、链接或类成员共享，三者不能互换。

回答要点

1. `const int*` 表示不能通过该指针修改整数，`int* const` 表示指针本身不能改指向。`const` 成员函数约束普通成员的修改，但 `mutable` 成员可变，指针成员所指对象也不是自动只读。
2. `constexpr` 变量要求常量表达式初始化；`constexpr` 函数只在满足常量表达式要求的调用中于编译期求值，其他调用仍可发生在运行期。用它表达可计算的常量，而非性能魔法。
3. 局部 `static` 对象具有静态存储期，首次执行到初始化时完成初始化；类 `static` 成员不属于某个实例。C++11 起局部静态初始化具有并发保障，但随后对对象的读写仍需同步。

常见追问

追问：`const_cast` 去掉 `const` 后能写吗？答：只有原始对象本身并非 `const` 时，通过去掉限定的路径写入才可能合法。若原始对象就是 `const`，强行写入是未定义行为；不能根据一次测试结果宣称可写。

易错点

`const` 不代表数据物理上放在只读区，`constexpr` 函数也不保证每次都在编译期执行。讨论 `static` 时先指出声明位置，才能给出准确含义。

延伸阅读 / 原始参考资料

ISO C++ 工作草案 • `cv` 限定

ISO C++ 工作草案 • 存储期

03. `new/delete` 与 `malloc/free` 有什么区别？

语言基础 / 基础

`new` 通常完成存储分配与对象初始化，`delete` 负责析构和释放；`malloc/free` 只管理原始存储。分配函数 `operator new` 与完整的 `new` 表达式也需区分。

回答要点

1. `new T` 先获取适合类型的存储，再执行初始化；`delete p` 对适用对象先调用析构，再释放存储。`malloc` 返回原始内存，不会替你调用类构造函数，`free` 也不会调用类析构函数。
2. 资源必须使用匹配的释放机制：`new` 对应 `delete`，`new[]` 对应 `delete[]`，`malloc` 对应 `free`。混用可能破坏分配器状态或漏掉数组元素析构，其后果不是简单的固定大小内存泄漏。
3. 默认抛异常的 `new` 分配失败通常报告 `bad_alloc`；`nothrow` 形式失败返回空指针。`placement new` 在已有位置构造对象，常用于内存池，生命周期和释放存储仍须由相应协议管理。

常见追问

追问：构造函数抛异常会泄漏吗？答：普通 `new` 表达式在构造失败时会调用匹配的释放函数归还本次存储，已成功构造的成员也会析构。但构造函数内部另行申请且未交给资源管理对象的资源仍可能泄漏。

易错点

不要把 `new` 当作 `malloc` 的语法糖。手动处理原始存储时同时考虑对齐、构造、析构与匹配释放；工程代码通常优先采用容器和智能指针。

延伸阅读 / 原始参考资料

ISO C++ 工作草案 • `new` 表达式

ISO C++ 工作草案 • `delete` 表达式

04. 虚函数如何实现多态？基类析构为什么常设为虚函数？

面向对象 / 基础

通过基类指针或引用调用虚函数时，会按实际对象类型选择最终覆盖函数。若要通过基类指针删除派生对象，基类通常必须有虚析构函数，才能按多态方式正确销毁。

回答要点

1. 动态分派依赖实际对象类型和虚函数覆盖关系；`override` 让编译器检查覆盖是否成立，避免参数或 `const` 不一致形成隐藏函数。显式限定 `Base::f` 的调用可以绕过普通虚函数分派。
2. 虚函数表和虚表指针是主流编译器的常见实现策略，但不是语言强制的内存布局。讨论对象大小、多继承或性能时需要说明编译器、平台 ABI，以及可能进行的去虚拟化优化。
3. 基类若允许外部多态删除，采用公开虚析构通常最直接；若不允许这样删除，也可采用受保护的虚析构限制接口。按值传递派生对象给基类参数会切片，无法保留完整派生状态。

常见追问

追问：纯虚函数能有实现吗？答：可以在类外提供定义，但类仍可能是抽象类，派生类需完成要求的覆盖。纯虚析构函数通常必须有定义，因为销毁派生对象时仍会执行基类析构链。

易错点

把“有虚函数就一定拥有固定一个虚表指针”当标准结论不准确。经基类指针删除时不能仅根据派生类是否有虚函数判断安全，应检查基类析构。

延伸阅读 / 原始参考资料

ISO C++ 工作草案 • 虚函数

ISO C++ 工作草案 • 析构函数

05. 构造和析构顺序是什么？构造函数能调用虚函数吗？

面向对象 / 基础

构造依次完成虚基类、直接基类、成员和构造函数体；成员按声明顺序初始化，析构大体反向进行。构造析构期间的虚调用受当前阶段限制，不能期待派生类行为已经存在。

回答要点

1. 最派生类负责虚基类初始化，直接基类按基类列表顺序构造，数据成员按类中声明顺序构造。初始化列表的书写次序不会改变成员顺序，因此依赖关系需要体现在成员声明顺序中。
2. 构造函数体执行前，相关基类和成员已经完成初始化。引用成员、const 成员和没有默认构造函数的成员通常需要在初始化列表中初始化，先默认构造再赋值会带来多余工作。
3. 在基类构造或析构过程中调用虚函数，分派到当前构造析构阶段对应的类，而不会调用尚未构造或已经销毁的派生部分。把多态工作放到对象完整构造后的明确操作中更易推理。

常见追问

追问：构造失败会调用完整对象析构吗？答：不会，因为完整对象尚未成功构造。但已构造完成的基类和成员会被销毁；所以将资源放进具有可靠析构函数的成员，能让构造失败路径自动回收。

易错点

容易将初始化列表顺序误认为真实执行顺序。还要警惕构造期间发布 this 给其他线程或回调，使外部观察到对象不完整的状态。

延伸阅读 / 原始参考资料

ISO C++ 工作草案 • 基类与成员初始化

ISO C++ 工作草案 • 构造析构期间的对象

06. 浅拷贝、深拷贝与 Rule of Zero/Five 怎么理解？

所有权与资源 / 基础

浅拷贝复制地址或句柄，深拷贝建立独立资源。管理资源的类必须明确拷贝和移动语义；优先让标准资源成员代管，遵循 Rule of Zero，而非为每个类手写五个特殊成员。

回答要点

1. 编译器生成的拷贝通常按成员进行，裸指针成员只复制指针值，不会复制所指资源。两个对象若都把同一指针当独占所有权，就可能重复释放；若都不释放，则可能泄漏。
2. 需要手动管理资源时，析构、拷贝构造、拷贝赋值、移动构造和移动赋值要作为整体审视。可以选择深拷贝，也可以删除拷贝并支持移动；五法则并不要求所有类都允许五种操作。

3. 采用 `string`、`vector`、`unique_ptr` 等成员，往往可以由它们实现销毁和转移。拷贝赋值还要处理自赋值和异常安全：先完成新资源，再提交替换，比先释放旧资源更容易保留原状态。

常见追问

追问：写了析构函数后还会自动生成移动吗？答：用户声明的析构函数会影响隐式移动特殊成员生成。应检查实际类型的生成规则，必要时显式 `default` 或 `delete`。仅看到 `std::move` 调用不能证明一定走移动构造。

易错点

不要对包含 `string` 或虚函数等复杂对象直接用 `memcpy` 完成语义拷贝。对象字节相同不等于拥有关系正确，特殊成员是否可用还取决于成员类型。

延伸阅读 / 原始参考资料

ISO C++ 工作草案 • 拷贝与移动构造

ISO C++ 工作草案 • 拷贝与移动赋值

07. RAII 为什么能改善异常安全？

所有权与资源 / 基础

RAII 把资源所有权绑定到对象生命周期：成功构造取得资源，析构释放资源。离开作用域或异常展开时，已经构造的局部管理对象会自动析构，减少遗漏释放的路径。

回答要点

1. 资源不仅是内存，也包括文件、锁、套接字和事务句柄。先设计一个明确拥有资源的对象，再通过作用域控制释放时机；让裸指针承担借用角色，可避免谁负责关闭资源的歧义。
2. 取得资源后应尽快交给管理对象；随后任一步骤抛异常，已建立的资源成员仍能析构。多个资源各自由成员管理，构造失败时语言会销毁已经完成构造的成员，降低清理分支数量。
3. 析构中应尽量使用不抛异常的释放操作，尤其不能在栈展开时继续让异常逃逸。RAII 保证资源清理，但自动保证业务操作原子性，强异常保证仍需要先准备、后提交的状态设计。

```
#include <mutex>
void update(std::mutex& mutex, int& value) {
    std::lock_guard<std::mutex> lock(mutex);
    ++value;
}
```

常见追问

追问：`lock_guard` 与 `unique_lock` 怎么选？答：只需要固定作用域加锁时使用 `lock_guard` 很直接；需要延迟加锁、提前解锁或与条件变量配合时可用 `unique_lock`。两者都表达锁的所有权，但接口能力与成本不同。

易错点

RAII 不保证进程被强制终止时一定执行析构，也不能代替事务回滚策略。它解决常规 C++ 控制流中的资源生命周期，仍需要正确表达资源拥有关系。

延伸阅读 / 原始参考资料

C++ Core Guidelines • R.1 RAII

ISO C++ 工作草案 • 异常中的构造与析构

08. `unique_ptr` 的所有权怎样转移？`release` 与 `reset` 有何不同？

所有权与资源 / 基础

`unique_ptr` 表达独占所有权，不能复制但可移动。`reset` 处理原资源并接管新资源，`release` 交出指针并放弃管理；调用 `release` 后必须落实新拥有者。

回答要点

1. 优先使用 `make_unique` 构造独占对象，把所有权转移写成 `std::move`。转移后的源 `unique_ptr` 为空；仅借用对象时传指针或引用即可，不必把所有调用都设计成智能指针参数。
2. `reset` 会用删除器销毁原来管理的对象，然后接管给定指针；`release` 返回原指针而不销毁对象。调用 `release` 后如果无人负责释放，就会泄漏，因此它主要用于接入明确的所有权接口。
3. `unique_ptr` 可以定制删除器以管理特定资源，但删除器类型也会影响对象大小和可移动性。
`unique_ptr<T[]>` 管理数组；转为 `unique_ptr<Base>` 并经默认删除器删除派生对象时要检查虚析构。

常见追问

追问：`unique_ptr` 能放进 `vector` 吗？答：可以，容器可通过移动转移元素所有权。插入已有指针通常用 `push_back(std::move(p))`，成功后 `p` 为空。不能直接复制一个 `vector<unique_ptr<T>>` 来获得独立副本。

易错点

`get` 得到的是借用地址，不能另建一个独占所有者再交给另一个 `unique_ptr`，也不能手动 `delete` 后保留原拥有者。两个所有者会导致重复释放。

延伸阅读 / 原始参考资料

ISO C++ 工作草案 • 独占智能指针

ISO C++ 工作草案 • 独占指针修改操作

09. `shared_ptr` 的控制块与线程安全边界是什么？

所有权与资源 / 进阶

`shared_ptr` 通过控制块共享对象所有权，最后一个强拥有者释放对象。共享控制块的不同指针实例可安全维护引用计数，但被指向对象和同一个指针实例的并发修改仍需额外同步。

回答要点

1. 控制块记录强拥有计数、弱引用关联以及删除方式等信息。`make_shared` 常见实现会将对象与控制块放在一次分配中，有利于减少分配，但弱引用仍存活时也可能延后整块存储回收。
2. 不同 `shared_ptr` 实例即使共享控制块，维护自身引用计数不等于保护对象内容。多个线程通过它们修改同一个普通对象仍会发生数据竞争；必须用互斥锁或对象自身的同步机制。
3. 不能从同一裸指针分别构造两个独立 `shared_ptr`，它们会建立独立的控制块并重复销毁资源。需要自引用时使用 `enable_shared_from_this`，并且对象已经进入 `shared_ptr` 管理后才能使用。

常见追问

追问：`use_count()==1` 可以据此无锁修改对象吗？答：不应把它当并发独占证明，计数可能马上变化，其他线程也可能保有借用地址。`C++20 atomic<shared_ptr<T>>` 可同步指针发布，但仍不会自动保护 `T` 的内部状态。

易错点

“`shared_ptr` 线程安全”必须说清保护范围。别混淆控制块、指针实例与目标对象三种不同共享状态；共享所有权也会增加计数维护和生命周期推理成本。

延伸阅读 / 原始参考资料

ISO C++ 工作草案 • 共享智能指针

ISO C++ 工作草案 • 原子共享指针

10. `weak_ptr` 怎样打破循环引用？为什么使用 `lock`？

所有权与资源 / 进阶

`weak_ptr` 观察 `shared_ptr` 管理的对象而不增加引用计数。将回指或观察关系改成弱引用，可以避免强所有权环；使用前通过 `lock` 原子地尝试取得临时强所有权。

回答要点

1. 若 `A` 和 `B` 互持 `shared_ptr`，外部拥有者消失后两者强计数仍非零，无法回收。先确定哪条边代表真正拥有关系，再将父节点回指或观察者边改为 `weak_ptr`，而非随意全部改弱引用。
2. `expired` 只报告检查时刻强计数是否为零，不能保证后续使用仍有效。`lock` 将检查与取得拥有权作为原子操作：成功返回有效 `shared_ptr`，失败返回空指针，成功期间目标对象保持存活。
3. `weak_ptr` 不让对象本身继续存活，但需要控制块来判断状态，控制块会保留到相关弱引用释放。对象销毁与控制块存储回收是两个时刻，不能简单认为弱引用完全不产生资源成本。

```
if (auto owner = weak.lock()) {
    // owner 存活期间才使用对象
    owner->work();
}
```

常见追问

追问：缓存为什么会使用 `weak_ptr`？答：缓存可以记录已存在对象但不主动延长其生命。查找时 `lock` 成功则复用，失败则重新构造并更新弱引用。并发场景还需要同步缓存索引，避免重复创建及结构数据竞争。

易错点

先调用 `expired` 再直接使用先前保存的裸指针，仍存在时间窗口。安全使用通常要检查 `lock` 的返回值，并持有得到的 `shared_ptr` 到访问结束。

延伸阅读 / 原始参考资料

ISO C++ 工作草案 • 弱智能指针

ISO C++ 工作草案 • 弱引用观察操作

11. `std::move`、右值引用和完美转发有什么联系？

现代 C++ / 进阶

`std::move` 转换值类别，本身不搬运资源；是否移动由重载解析和类型能力决定。完美转发以转发引用和 `std::forward` 保留值类别，使包装函数正确选择下游重载。

回答要点

1. 左值通常表示有身份的对象表达式，右值包含纯右值和将亡值。具名的 `T&&` 参数表达式本身仍是左值；`std::move` 把表达式转为将亡值，随后才可能匹配移动构造或移动赋值。
2. 移动通常转移底层资源，但移动后的标准库对象一般只保证有效、状态未指定，除非类型有更强约定。不能统一断言所有 `string` 或 `vector` 移动后都为空，也不能无条件继续读取业务值。
3. 模板推导中的 `T&&` 可形成转发引用，配合引用折叠和 `std::forward<T>` 保留左值右值属性。普通固定类型的 `T&&` 不自动成为转发引用；移动 `const` 对象通常仍会走拷贝，因为常见移动接口要修改源。

常见追问

追问：`return std::move(local)` 总比 `return local` 快吗？答：不一定，显式 `move` 可能阻碍命名返回值优化。优先直接 `return local`，让编译器使用可选 NRVO 或隐式移动；C++17 的部分纯右值返回场景还有保证的拷贝消除。

易错点

把 `std::move` 翻译成“立即移动”会误判 `const`、无移动函数和重载情况。移动构造的 `noexcept` 属性也会影响容器重分配时是否选择拷贝。

延伸阅读 / 原始参考资料

ISO C++ 工作草案 • `move` 与 `forward`

ISO C++ 工作草案 • 拷贝消除

12. 模板、重载、特化与 C++20 concepts 怎么区分？

现代 C++ / 进阶

模板描述可按类型或值实例化的通用代码，重载为同名调用提供候选函数，特化为特定模板参数提供实现。C++20 concepts 用约束表达模板需求，使接口意图和错误定位更明确。

回答要点

1. 模板不是运行期解释器，编译器会在需要时进行推导与实例化。模板定义通常放在头文件，使使用位置能看到定义；特定场景也可通过显式实例化把支持的参数集合集中到源文件。
2. 函数模板可以重载并进行全特化，但不能像类模板那样直接做偏特化。实现部分类型差异时通常采用函数重载、类型特征或类模板偏特化，要区分候选选择与模板实例化两个阶段。
3. SFINAE 会在特定替换失败场景移除候选，不能把所有模板错误都当作替换失败。concepts 和 requires 提供更直接的约束表达；它们检查声明需求，不替你证明算法的业务正确性。

常见追问

追问：模板为什么可能增大二进制？答：不同参数可能实例化不同代码，过多组合增加编译时间和体积。可以把与类型无关的工作提取到普通函数，或采用有限显式实例化；同时也应让编译器优化与实际测量指导取舍。

易错点

不要把模板等同于宏：模板仍受类型系统与作用域规则约束。泛型代码的约束、实例化位置与重载规则需要一起解释，不能只背一个 template 语法。

延伸阅读 / 原始参考资料

ISO C++ 工作草案 • 模板

ISO C++ 工作草案 • 模板约束

13. lambda 的值捕获、引用捕获与生命周期怎样判断？

现代 C++ / 基础

lambda 生成一个闭包对象，捕获决定它保存外部状态的方式。值捕获通常保存副本，引用捕获借用原对象；闭包被异步保存时，必须确保它借用的对象仍然活着。

回答要点

1. 值捕获在创建闭包时获取当时的值，默认调用运算符对捕获成员呈 const 访问；mutable 允许修改闭包中的副本，但不表示外部原变量被修改。引用捕获则通过借用访问原状态。
2. 捕获 this 保存的是对当前对象的访问关系，闭包不会因此自动拥有对象。C++17 的 [*this] 可复制对象本身，但复制的内部裸指针也不会自动深拷贝其资源，需要检查成员拥有语义。

3. 初始化捕获可以把 `unique_ptr` 移入闭包，明确延长所管对象生命。返回闭包或投递线程任务时，优先显式列出捕获对象；按引用捕获局部变量后让闭包逃离作用域会造成悬空访问。

常见追问

追问：捕获 `unique_ptr` 的 `lambda` 能放入 `std::function` 吗？答：在 C++17/20 常用 `std::function` 中，目标通常要求可复制，因此仅有移动能力的闭包不满足要求。可直接用模板参数承载闭包，或重新设计共享所有权，不能盲目复制。

易错点

默认 `[&]` 在同步短作用域内便利，在异步回调里却易隐藏借用关系。判断安全时先写出闭包的存活时间，再检查每个捕获对象的有效期。

延伸阅读 / 原始参考资料

ISO C++ 工作草案 • `lambda` 表达式

ISO C++ 工作草案 • `std::function` 构造

14. `vector` 为什么适合默认使用？扩容与失效规则是什么？

STL 与容器 / 基础

`vector` 提供连续存储与常数时间随机访问，尾部追加为摊还常数时间。发生重分配时，指向元素的引用、指针与迭代器都失效；没有重分配时也要按插入删除位置判断有效性。

回答要点

1. `size` 表示已经存在的元素数量，`capacity` 表示当前存储可容纳的数量。`reserve` 提前规划容量但不会增加 `size`，也不能把未构造容量当作现存元素访问；`resize` 才会改变实际元素个数。
2. 尾部追加可偶尔触发线性迁移，因此是摊还 $O(1)$ ，单次不保证常数。标准不规定固定两倍增长倍率；中间插入删除需要移动后方元素，即使容量充足也通常是线性成本。
3. 重分配会让所有元素地址和迭代器失效；没有重分配的插入仍会让插入点及其后迭代器失效，`erase` 也会影响删除点及之后。保存下标有时更方便，但插删仍可能改变逻辑对应关系。

常见追问

追问：`reserve` 后能保证所有地址不变吗？答：只要后续大小不超过预留容量，尾部追加可避免该阶段重分配；中间插删仍可能移动元素。调用 `reserve` 请求更大容量本身也可能重分配，缓存地址必须在正确时机更新。

易错点

`vector<bool>` 是特殊化，不应套用普通 `vector<T>` 的真实 `bool` 引用和连续 `bool` 对象假设。容器迁移策略还依赖元素拷贝、移动和异常保证。

延伸阅读 / 原始参考资料

ISO C++ 工作草案 • vector 规范
ISO C++ 工作草案 • vector 修改操作

15. list、deque 与 vector 如何选？

STL 与容器 / 基础

先按操作需求选择容器：vector 连续存储和随机访问强；deque 支持两端插入及随机访问但不承诺整体连续；list 支持已知位置的节点操作，却缺少随机访问且缓存局部性较差。

回答要点

1. list 的插入删除在已知迭代器位置通常为常数时间，并能保留未删除节点的迭代器与引用。若先按下标查找位置，遍历已经需要 $O(n)$ ，不能把整个业务操作描述为常数时间。
2. deque 通常采用分段结构，两端增长不要求搬迁全部元素；语言接口保证随机访问，但不能将整段数据当作连续数组传给 C 接口。两端插入的迭代器失效规则与元素引用规则也不同。
3. vector 通常是读多、遍历多场景的良好默认选择，连续存储可减少指针跳转与分配开销。list 的节点额外空间、分配成本和实际缓存表现都要计入，不能只看插入删除的渐进复杂度。

常见追问

追问：deque 尾部插入会让引用失效吗？答：两端插入会使迭代器失效，但不影响既有元素引用有效性；中间插入则会使迭代器与引用都失效。不能将同一个容器的迭代器与引用稳定性混为一谈。

易错点

“链表插入快”省略了定位成本和分配成本。选择容器应根据实际访问模式、地址稳定性要求和测量结果，而不是根据某个复杂度单项排名。

延伸阅读 / 原始参考资料

ISO C++ 工作草案 • list 规范
ISO C++ 工作草案 • deque 规范

16. map 和 unordered_map 的性能与键规则有什么差别？

STL 与容器 / 基础

map 维护按比较器定义的键顺序，查找通常为对数时间；unordered_map 通过哈希组织数据，查找平均常数、最坏线性。是否需要范围查询、稳定性能和遍历顺序是选择的重要依据。

回答要点

1. map 的比较器应满足严格弱序，键等价由双方都不小于对方判定，不必等同于 `operator==`。常见实现是红黑树，但标准关注接口、复杂度和失效保证，不强制这一具体树结构。

2. `unordered_map` 要求相等键具有相同哈希值；良好的散列分布和合适负载因子支持平均常数查找。恶意碰撞或糟糕哈希可导致线性退化，还需考虑字符串键的哈希计算成本。
3. `unordered_map` 重哈希通常使迭代器失效，但不使元素引用和指针失效。`operator[]` 在键不存在时会插入并默认构造值；仅想查询时可用 `find`，避免查询行为意外修改容器。

常见追问

追问：查找最快就总选哈希表吗？答：有序遍历、`lower_bound` 或键区间检索更适合 `map`；哈希表还可能有额外桶空间及碰撞退化。确定访问模式后再比较，不应把平均 $O(1)$ 当作任何数据上的确定低延迟。

易错点

不要在容器存储期间改变会影响排序或哈希等价的键状态。自定义比较器与哈希函数还应覆盖边界值，并且保持等价关系的一致性。

延伸阅读 / 原始参考资料

ISO C++ 工作草案 • 有序关联容器要求

ISO C++ 工作草案 • 无序关联容器要求

17. 迭代器类别、算法要求与删除遍历怎样关联？

STL 与容器 / 进阶

迭代器把容器访问方式交给算法，但不同类别支持不同操作。调用前既要满足算法的迭代器要求，也要遵守修改容器后的失效规则；删除遍历应使用删除操作返回的新位置继续。

回答要点

1. 输入迭代器主要支持单遍读取，前向迭代器可多遍遍历，双向迭代器支持退后，随机访问迭代器支持位移与距离。不能因为容器能遍历，就认为任意算法或 `it+n` 表达式都适用。
2. `std::sort` 要求随机访问迭代器，因此 `vector` 和 `deque` 常可使用，而 `list` 应使用自身 `sort` 成员。比较器必须满足严格弱序；用 `<=` 写比较器会让相等值也互相小于，破坏算法前提。
3. 遍历时删除元素不要自增已经失效的迭代器，通常以 `it=container.erase(it)` 继续。C++17 可用 `erase-remove` 模式清理序列；C++20 的 `erase_if` 简化部分容器的条件删除接口。

```
for (auto it = values.begin(); it != values.end(); ) {
    if (*it < 0) it = values.erase(it);
    else ++it;
}
```

常见追问

追问：`lower_bound` 在 `list` 上也是 $O(\log n)$ 吗？答：比较次数可以是对数，但移动迭代器的次数可能是线性的，整体时间不能按随机访问数组理解。有序关联容器优先使用其成员 `lower_bound`，让搜索利用树结构。

易错点

不要把 `end` 当作有效元素解引用。算法比较次数与实际迭代器移动成本不是同一指标，判断复杂度时需写明容器和算法接口。

延伸阅读 / 原始参考资料

ISO C++ 工作草案 • 迭代器要求

ISO C++ 工作草案 • 排序与二分算法

18. 异常安全的基本、强与不抛保证怎么区分？

可靠性与并发 / 进阶

基本保证要求失败后仍维持有效状态且不泄漏资源；强保证要求失败后可观察状态保持原样；不抛保证要求操作不让异常逃逸。异常安全要围绕状态提交与资源生命周期设计。

回答要点

1. 基本保证不等于没有任何变化，只保证对象不变量和资源管理仍可靠。强保证可以理解成成功则提交、失败则回滚；例如先把新数据建在临时对象中，全部成功后再以不抛的交换提交。
2. 栈展开会析构已经构造的自动对象，是 RAII 发挥作用的机制。析构若在异常展开过程中再抛出并逃逸，会导致终止；资源释放应尽可能不抛，失败信息可通过单独操作或其他通道报告。
3. `noexcept` 是对异常逃逸行为的承诺，不会把内部所有调用变成不会失败的调用。若异常逃出 `noexcept` 函数，会调用 `terminate`；条件 `noexcept` 可依据成员操作能力描述类型的移动和交换属性。

常见追问

追问：`vector` 为什么可能不用移动而改用拷贝？答：当移动可能抛异常而类型可拷贝时，重分配过程可能选择拷贝，以便失败后保留旧元素。若类型不可拷贝且移动抛出，部分操作的强保证会受限制，需要查具体规范。

易错点

`catch(...)` 后忽略错误并不自动提供强保证，数据可能已经部分修改。不要为追求表面无异常而随意加 `noexcept`，应先验证内部操作与终止语义是否符合需求。

延伸阅读 / 原始参考资料

ISO C++ 工作草案 • 异常处理

ISO C++ 工作草案 • 异常说明

19. 未定义行为、越界、对齐与类型别名怎样理解？

可靠性与并发 / 进阶

未定义行为表示语言不再约束该次执行的结果，不能用一次正常输出证明安全。越界、悬空访问、部分不合法类型访问和有符号溢出都是典型风险，必须从语言规则判断。

回答要点

1. 数组下标要求落在有效元素范围内；尾后指针可用于比较和终止遍历，但不能解引用。已经释放的存储可能仍保留旧字节，这不能恢复对象生命周期，也不能让读取重新变得合法。
2. 不同类型对象对存储的对齐要求不同，把任意字节地址强转成某类型指针并解引用可能同时违反对齐、生命周期与类型访问规则。`reinterpret_cast` 只改变解释方式，不负责建立有效对象。
3. 不要依靠互不相关的类型指针读取同一对象表示来做常规类型转换。检查位表示可用允许的字节访问方式；需要表示层转换时可针对适用类型使用 `memcpy` 或 C++20 `bit_cast`，并满足相关前提。

常见追问

追问：未定义行为是不是必定崩溃？答：不是，可能暂时正常、输出错误或被优化后出现难以解释的结果。调试构建与优化构建差异也很常见。使用地址与未定义行为检查工具辅助发现问题，但工具未报错不能代替证明合法。

易错点

“机器允许未对齐读”不等于 C++ 规则允许该类型访问。把硬件现象、编译器扩展和语言保证分开说明，避免用平台巧合回答跨平台问题。

延伸阅读 / 原始参考资料

ISO C++ 工作草案 • 值类别与类型访问

ISO C++ 工作草案 • 对齐要求

20. 数据竞争、atomic、mutex 与内存序是什么关系？

可靠性与并发 / 进阶

并发访问同一内存位置，至少一方写入且缺少适当同步，可能构成数据竞争并导致未定义行为。`atomic` 保护单项原子操作，`mutex` 可保护复合不变量，内存序决定跨线程可见性关系。

回答要点

1. `atomic` 读写解决该对象的原子访问，不能自动保护旁边的普通对象。即使两个字段分别是 `atomic`，也不能保证读取者看到的是同一版本组合；维护多个字段的一致性常用同一互斥锁。
2. `relaxed` 保证原子性和该原子对象的修改顺序，但不建立一般发布关系。`release` 写与读取到对应值的 `acquire` 读可以建立同步关系，使发布前写入的数据对接收线程可见，前提是协议完整。
3. 顺序一致序通常更易推理，但原子操作不保证无锁，也不保证算法无阻塞。`volatile` 不是线程同步机制；避免数据竞争还需要明确对象生命期、锁顺序和所有参与线程的访问约定。

```
#include <atomic>
std::atomic<int> count{0};
void record() {
    count.fetch_add(1, std::memory_order_relaxed);
}
```

常见追问

追问：i++ 为什么不能靠普通 int 完成并发计数？答：它包含读、修改和写，多个线程会互相覆盖且构成数据竞争。用 atomic 的 fetch_add 可完成原子计数；若计数还控制其他数据发布，应同时设计适当的内存序与生命周期协议。

易错点

不要只背 acquire/release 名称，要指出哪个发布与哪个读取同步。原子计数正确也不意味着整个业务操作正确，多步检查再修改仍可能需要锁。

延伸阅读 / 原始参考资料

ISO C++ 工作草案 • 数据竞争

ISO C++ 工作草案 • 原子内存序